Chapter 3

Research Methodology

3.1. Overview of the Proposed Framework

This study follows a systematic, reproducible methodological framework specifically designed for highly incomplete, mixed-type, real-world social datasets. The complete pipeline consists of six major phases:

Data Collection and Expansion

Intensive Preprocessing and Conditional Filling

Advanced Feature Encoding

Multiple Imputation Strategies (three imputation methods × two predictor-selection approaches)

Clustering Using Multiple Algorithms

Comprehensive Evaluation and Comparison

Through this pipeline, a total of eight datasets were generated—six fully imputed datasets and two non-imputed baseline datasets. Each dataset was clustered using four distinct clustering algorithms, producing 32 clustering solutions in total. These solutions were subsequently evaluated and ranked using a composite internal validation scoring framework.

3.2. Dataset Analysis

3.2.1. Data Collection and Expansion

The base dataset was obtained from Kaggle [4], containing suicide-related records from Bangladesh:

Source: Kaggle – Bangladesh Suicide Dataset

Initial size: 760 records × 27 columns

Coverage: Year 2020

To enhance temporal coverage and analytical richness, 857 additional recent cases were manually collected. The dataset was expanded to cover incidents from 2017 to 2025. These records were compiled from verified national and local Bangladeshi news outlets, including Prothom Alo, The Daily Star, Jugantor, Ittefaq, and multiple regional newspapers and online portals.

To capture socio-economic and clinical dimensions absent in the original dataset, nine new attributes were introduced:

- `family_members`
- `position_in_siblings`
- `marital_status`
- `duration_of_separation`
- `profession_description`
- `workplace`
- `financial_condition`
- `previous_health_condition`
- `previous_suicide_attempt`

After expansion, the final dataset statistics were:

Total records: 1,617

Total columns: 36 (before the ethical removals described in the next subsection; the analytical dataset used downstream contains 34 columns)

3.2.2. Removal of Irrelevant and Sensitive Columns

To address ethical concerns and analytical redundancy, the following columns were permanently removed:

full_name: removed to avoid the use of personally identifiable information

data_source: removed as the news URL did not contribute to clustering or analysis

3.2.3. Missing Value Analysis (Pre-Imputation)

Before applying any imputation, an extensive analysis of missing values was conducted. Features were grouped into four categories based on their percentage of missing values:

Group	Missing Range	Key Columns
1	77.61% - 96.17%	duration_of_separation, position_in_siblings, previous_suicide_attempt, previous_health_condition, family_members
2	45.51% - 53.06%	Weather-related variables (temp_min, temp_max), latitude, longitude, profession_description, financial_condition etc
3	23.93% - 37.90%	workplace, marital_status
4	0.43% - 18.92%	age, gender, method, reason, hometown, etc

All Group 1 variables (missing rate >70%) were permanently excluded from further analysis, as imputing such highly sparse features would introduce excessive uncertainty and noise [31].

3.3. Detailed Preprocessing Pipeline

3.3.1. Column Standardization & Type Correction

The dataset contained inconsistencies in naming conventions, spelling, and data types. The following corrective actions were applied:

Converted all column names to snake_case

Corrected spelling errors (e.g., maritual_status → marital_status)

Converted numerical values stored as strings to appropriate integer or float types

3.3.2. Date & Time Standardization

The suicide_date column existed in multiple formats (string, datetime, and numeric). Additionally, the time column was categorical (morning, noon, evening, night). To ensure consistency:

All dates were standardized to DD/MM/YYYY format

Categorical time values were mapped to representative hours (0–23)

Five temporal features were derived:

suicide_day

suicide_month

suicide_year

suicide_week

suicide_weekday

3.3.3. Categorical Value Harmonization (Manual + Semi-automated)

Categorical variables exhibited extensive inconsistencies due to spelling variations, synonym usage, casing differences, and over-granular labels. The number of unique categories ranged from 7 (gender) to 594 (hometown).

A column-wise iterative cleaning process was applied:

Spelling variants and synonyms were merged

Similar categories were grouped into broader, semantically consistent classes

The hometown feature was simplified to district-level labels only to reduce sparsity and address ethical concerns

This process was repeated iteratively for each categorical column until no duplicate semantic categories remained. After harmonization, the highest cardinality column (hometown) was reduced to 151 unique categories.

3.3.4. Conditional Filling Using External APIs

To reduce missingness without arbitrary imputation, external data sources were conditionally integrated:

Geolocation (Latitude & Longitude): Using the cleaned hometown field and the Geopy geocoding service, missing latitude and longitude values were filled. Missingness was reduced from ~53% to 6.49% for both variables.

Unix Time Construction: Using suicide_date and encoded time, a unix_time feature was computed, reducing missingness from 52.38% to 5%.

Weather Variables: Weather attributes were conditionally filled using the Meteostat API, leveraging the unix_time, latitude, and longitude. Hourly data were prioritised; daily aggregates were used when hourly data were unavailable.

Column	Before	After	Status
temp_min	53.06%	29.87%	Reduced
temp_max	53.06%	29.56%	Reduced
air_pressure	53.06%	30.43%	Reduced
air_humidity	53.06%	53.06%	No Change
wind_speed	52.99%	30.18%	Reduced
wind_deg	53.06%	53.06%	No Change

Values for air_humidity and wind_deg didn't change because the data for these columns for the given dates and location were not present in API.

No values were filled for feels_like, clouds_sky, weather_main, and weather_description due to the unavailability of data in the API for the given spatiotemporal conditions.

3.3.5. Feature Engineering & Final Cleaning

The following steps were applied before encoding:

Dropped long free-text columns (profession_description, reason_description)

Removed suicide_date and unix_time after feature extraction

Final feature set: 29 columns (13 categorical, 16 numerical)

3.3.6. Encoding Strategy

To convert categorical variables into numerical representations suitable for imputation and clustering, four encoding techniques were applied based on feature characteristics:

Ordinal Encoding:

Certain categorical variables exhibit an intrinsic and meaningful order among their categories. For such variables, ordinal encoding was employed to preserve relative ordering information in numerical form. This encoding was applied to the following features:

age_group

financial_condition

weather_main

suicide_weekday

The mappings were manually defined based on domain knowledge to ensure semantic correctness. For example, the age_group feature was encoded as: {baby=0, child=1, teen=2, young=3, adult=4, middle-aged=5, old=6}

One-Hot Encoding:

Low-cardinality nominal variables without inherent ordering were encoded using one-hot encoding, which converts each category into a separate binary indicator without imposing a spurious ordinal relationship between values [32]. This approach was applied to the following features:

gender

workplace

Religion

Frequency (Proportional) Encoding:

High-cardinality categorical variables without ordinal structure were encoded using proportional frequency encoding. This method was applied to:

profession_group

hometown

reason

method

weather_description

marital_status

Applying one-hot encoding to these features would result in a highly sparse feature space, which could lead to the curse of dimensionality and adversely affect clustering performance.

Therefore, proportional frequency encoding was chosen over raw count encoding due to its Efficient Handling of High Cardinality, preserving meaningful patterns, and improving model interpretability [33].

Formally, for a categorical feature f and a category c, the proportional frequency encoding value $\phi(c,f)$ is defined as:

$$\phi(c,f) = \frac{n_{c,f}}{N_f}$$

where $n_{c,f}$ is the number of occurrences of category c in feature f, and N_f is the total number of non-missing observations in feature f.

Encoded values were stored in newly created columns to preserve the original categorical variables for interpretability and post-clustering analysis.

Numerical Features:

All numerical variables were retained in their original form, as they were already represented in appropriate integer or floating-point formats.

After applying all encoding procedures, the final dataset consisted of 48 numerical features, making it fully prepared for downstream imputation and clustering.

Label Encoding (special case):

For the special case of imputing the dataset with MissForest, all of the columns were encoded with the numeric labels using the LabelEncoder from scikit-learn.

Decoding preparation:

The encoding mappings for all categorical variables were saved in JSON files to enable consistent decoding after clustering, facilitating interpretation of results, cluster profiling, and visualisation of real-world patterns.

3.4. Imputation Strategies

Missing values in the dataset, stemming from incomplete newspaper reports on suicide cases, were handled through multivariate imputation to retain sample size, reduce bias, and preserve the integrity of features such as demographics (e.g., age, gender, profession_group), spatiotemporal factors (e.g., latitude, longitude, suicide_date), and environmental variables (e.g., temperature, weather_main). Three established imputation methods were selected: Multiple Imputation by Chained Equations (MICE), K-Nearest Neighbours (KNN), and MissForest. These methods were chosen to span a range of approaches—parametric and model-based (MICE), non-parametric and instance-based (KNN), and ensemble tree-based (MissForest)—enabling a comprehensive comparison of their performance on mixed-type data with varying missingness patterns. This multi-method strategy aligns with best practices for robust imputation evaluation, as no single technique is universally optimal.

For each method, two predictor selection strategies were applied, resulting in six variants. Parameters were standardized where possible (e.g., 10 iterations for convergence in iterative methods, k=5 for KNN) to ensure fair comparisons. All imputations were implemented in Python using libraries such as scikit-learn (for MICE via IterativeImputer with BayesianRidge estimator, KNNImputer) and a custom MissForest-equivalent implementation built on scikit-learn's random-forest estimators. Post-imputation, residual missing values were $\leq 0.25\%$ across all variants, yielding effectively complete datasets.

3.4.1. Common Functionalities of All Three Methods

3.4.1.1. Predictor Selection Strategies

Two strategies were employed for predictor selection to cover both selective and inclusive scenarios, allowing evaluation of their impact on imputation quality.

3.4.1.1.1 Correlation-based (Context-aware Selection)

For each target column, pairwise Pearson correlation coefficients were computed with all other columns using pandas' corr() function (defaulting to Pearson correlation). Self-correlations (always 1.0) were excluded, and only non-zero correlations ($|r| > 0.0$) were retained. A sorted dictionary of predictors was created, ordered by descending absolute correlation value. Only these correlated columns were used as predictors, focusing on informative relationships to reduce noise, multicollinearity, and computational burden while enhancing imputation quality.

3.4.1.1.2 All-available (No-context Selection)

This baseline used all columns in the dataset (except the target column itself) as predictors, maximising available information but potentially including weakly related or redundant variables.

3.4.1.2. Constraints

To prevent unrealistic imputed values (e.g., negative temperatures or latitudes outside Bangladesh), post-prediction constraints were applied to selected numerical, temporal, and encoded columns. Constraints were derived from domain knowledge, geographical facts, historical weather records, and logical limits. Examples include age (0–100 years), latitude and longitude (Bangladesh-specific geographic bounds), temperature-related columns (feels_like, temp_min, temp_max: plausible regional Kelvin range), wind_deg (0–360°), and encoded ordinal columns (e.g., suicide_weekday_encoded: 0–6). Values were rounded (for integers) and clipped to these bounds.

3.4.1.3. Functions Overview (Generalized Workflow)

Custom Python functions processed the imputation column-by-column and took as input the target column_name, DataFrame df, predictor selection dictionary (context-aware or no-context), and constraints dictionary.

The common workflow was as follows:

Identify rows with missing values in the target column.

For each missing row, dynamically select usable predictors (non-null in that row).

If no usable predictors exist, skip the row.

Prepare training data from complete cases in df (rows where target and selected predictors are non-null).

Train an appropriate model and predict the missing value (missing predictors temporarily filled with column mean for numerics or mode for categoricals).

Apply rounding (if needed) and clipping to constraints.

Update the DataFrame with the imputed value.

This shared structure ensured consistency across methods, with differences arising from model type and encoding compatibility. A visual overview of the workflow is provided in Figure 3.1 (with branches for context-aware vs. no-context predictor selection).

3.4.2. Method-Specific Adaptations

3.4.2.1. Multiple Imputation by Chained Equations (MICE)

MICE is a flexible, iterative parametric method that models each variable conditionally on the others using chained regression equations, generating multiple plausible imputations to account for uncertainty [9]. It is particularly suitable for mixed-type data under MAR assumptions.

In this implementation, MICE used scikit-learn's IterativeImputer with a BayesianRidge estimator (10 iterations, 5 imputations). Both context-aware and no-context variants followed the generalized workflow, with MICE supporting mixed encodings (one-hot, frequency, ordinal) natively.

3.4.2.2. K-Nearest Neighbours (KNN)

KNN is a non-parametric instance-based method that fills missing values by averaging (or mode for categoricals) from the k most similar complete observations based on distance metrics [10]. It is simple, fast, and robust to local patterns.

Here, scikit-learn's KNNImputer was used (k=5, uniform weights). The method supported mixed encodings and followed the same generalized workflow for both predictor strategies.

3.4.2.3. MissForest

MissForest is a non-parametric iterative method that uses random forests to impute missing values, capturing non-linear interactions without distributional assumptions [11]. It was applied with 100 trees per forest (`n_estimators=100`) and a fixed random state for reproducibility.

Label encoding was preferred for categoricals to optimise tree-based performance (unlike mixed encoding for MICE/KNN). Imputed categorical values were rounded and clipped to valid integer ranges post-imputation to match the original encoding.

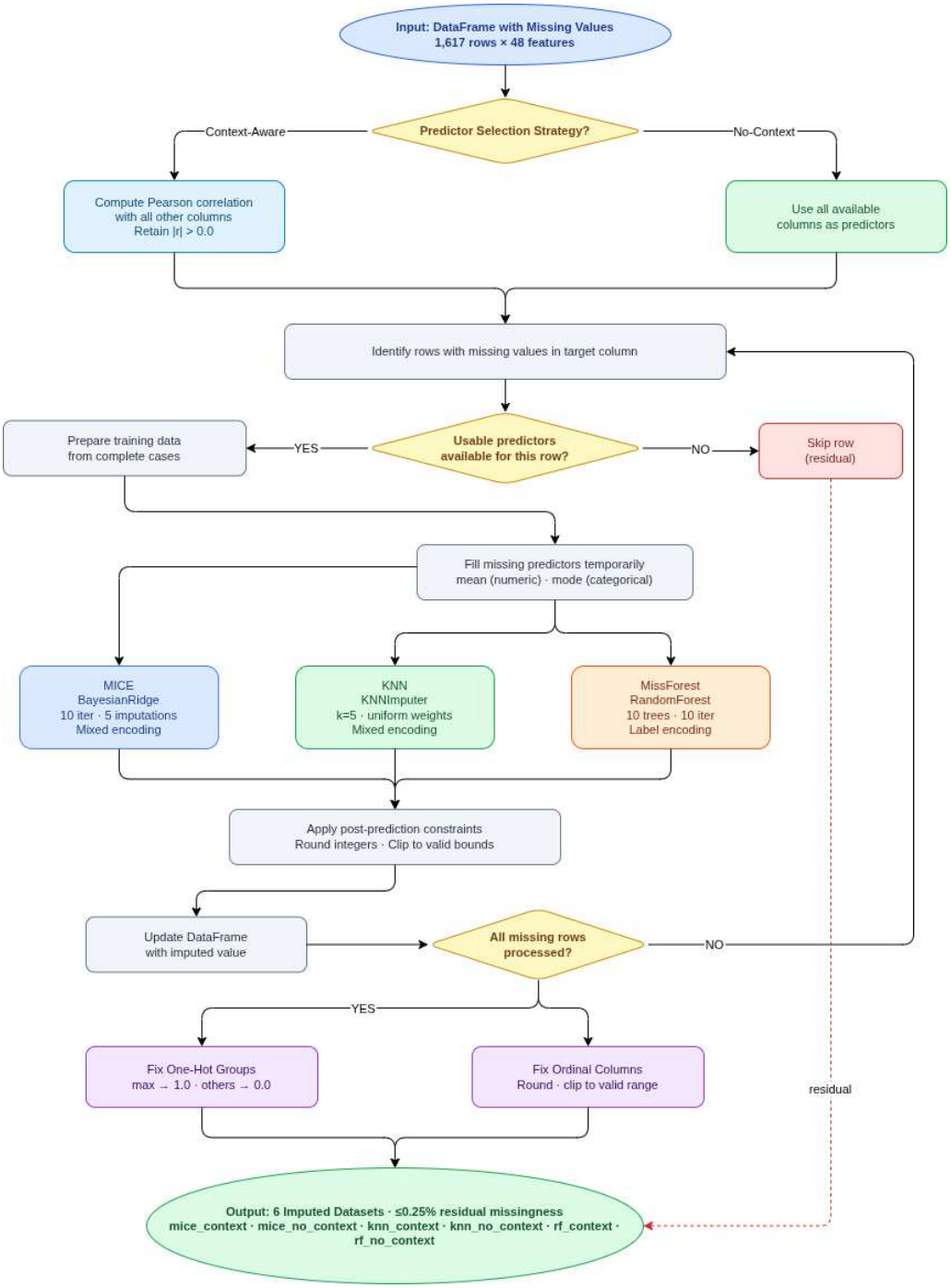

Figure 3.1: Unified imputation workflow for MICE, KNN, and MissForest (context-aware vs. no-context variants).

3.4.3. Post-Imputation Processing

After imputation, processing ensured categorical integrity and was identical across all variants.

3.4.3.1. Fixing One-Hot Encoded Columns

One-hot encoded groups (e.g., gender: female/male/third gender; workplace: rural/semi-urban/urban; religion: buddhist/christian/hindu/indigenous/muslim) sometimes produced fractional or multiple active values. These were corrected by setting the column with the highest imputed value to 1.0 and all others in the group to 0.0 per row.

3.4.3.2. Fixing Ordinal Encoded Columns

Ordinal columns (e.g., suicide_weekday_encoded: 0–6; age_group_encoded: 0–6; financial_condition_encoded: 0–4; weather_main_encoded: 0–7) were rounded from floats to nearest integers and clipped to valid bounds to restore semantic meaning.

3.4.4. Summary of Imputation Variants

The table below summarises the six variants, including abbreviations and missingness reduction (initial missingness varied by column; post-imputation $\leq$0.25% for all).

Imputation Method	Predictor Strategy	Abbreviation Used	Notes On Implementation
MICE	Correlation-based context	mice_context	BayesianRidge, mixed encoding
MICE	All available columns	mice_no_context	BayesianRidge, mixed encoding
KNN	Correlation-based context	knn_context	KNNImputer, mixed encoding
KNN	All available columns	knn_no_context	KNNImputer, mixed encoding
MissForest	Correlation-based context	rf_context	RandomForest, label encoding + rounding
MissForest	All available columns	rf_no_context	RandomForest, label encoding + rounding

3.5. Clustering Algorithms

3.5.1. Dataset Preparation for Clustering

The clustering analysis was performed on 8 distinct datasets to enable a comprehensive comparison between imputed and non-imputed versions. The 6 imputed datasets were generated from the three imputation methods (MICE, KNN, MissForest) using two predictor selection strategies (context-aware and no-context); the MICE and KNN variants retained 1,613 rows each (after dropping a small residue of rows that could not be imputed under their convergence criteria), while the MissForest variants retained the full 1,617 rows. All six datasets contained 37 columns after imputation.

For baseline comparison, a non-imputed version (df_encoded) was created by applying manual imputation, logical fillings, and encoding to the original data, resulting in varying missingness levels across columns. A missing data report revealed no columns with >53.06% missing, but significant missingness in groups: >45.51% to ≤53.06% (e.g., feels_like, air_humidity at 53.06%), >23.93% to ≤37.90% (e.g., workplace_encoded at 37.90%), and ≤18.92% (e.g., profession_group_encoded at 18.92%).

To create complete baseline datasets, rows with any missing values were dropped. Three variants were formed based on missingness thresholds:

df_encoded_20: Columns with ≤18.92% missing (853 rows, 22 columns)

df_encoded_40: Columns with ≤37.90% missing (403 rows, 30 columns)

df_encoded_50: All columns (≤53.06% missing, but effectively all since max was 53.06%; 57 rows, 37 columns)

The df_encoded_50 was excluded due to insufficient rows (57) for meaningful clustering. Thus, the final 8 datasets were: df_encoded_20, df_encoded_40, and the 6 imputed versions. These were compiled into a collection for batch processing, ensuring consistent application of clustering pipelines.

3.5.2. Overview and Rationale

Clustering was applied as an unsupervised technique to discover hidden patterns in the suicide dataset, where no predefined target variable existed for prediction. Four diverse algorithms were selected to cover different clustering paradigms: centroid-based (K-Means), hierarchical (Agglomerative Clustering), density-based (DBSCAN), and graph-based (Spectral Clustering). This selection provided robustness, as each method handles data structures differently—K-Means works with neighbours and means, Agglomerative uses hierarchical merging, DBSCAN focuses on density, and Spectral Clustering employs graph-based partitioning.

The goal was to evaluate performance across the 8 input datasets (6 imputed + 2 baselines), resulting in 32 combinations, using a combined scoring system to identify the best

pipeline. This allowed assessment of how imputation strategies influenced cluster quality and revealed real-world suicide patterns (e.g., demographic or environmental groupings), proving the pipeline's utility.

3.5.3. Common Preprocessing and Evaluation

All clustering methods shared a standardized preprocessing and evaluation pipeline to ensure fair comparison, with parameters tuned dynamically via grid search or fine-tuning.

3.5.3.1. Preprocessing Pipeline

Data was preprocessed to normalise features and reduce dimensionality. Numerical columns were scaled using StandardScaler (mean 0, std 1), ensuring equal contribution to distance metrics. Principal Component Analysis (PCA) was applied to compress high-dimensional data while retaining significant variance (e.g., 99% variance or fixed n_components like 10 or 20), improving efficiency and reducing noise while preserving key patterns.

3.5.3.2. Optimal Cluster Number Determination

The optimal number of clusters (k) was determined using silhouette analysis (measuring cluster cohesion/separation) and the elbow method (plotting inertia/distortion vs. k to find the "elbow" point). Tested ranges varied by method (e.g., 2–20 for K-Means, 2–11 for Agglomerative, 2–6 for Spectral), applied per dataset to adapt to varying input structures.

3.5.3.3. Hyperparameter Tuning Strategy

Hyperparameters were optimised using grid search followed by fine-tuning, with subset sampling (e.g., 1000 rows) for computational efficiency on large datasets. Performance was evaluated using the silhouette score (higher better), the Calinski-Harabasz index (higher better), and the Davies-Bouldin index (lower better). These were combined into a method-specific composite score for selection of the best parameter set. Parallel processing was utilised where applicable to accelerate the search.

3.5.4. Method-Specific Adaptations

3.5.4.1. K-Means (Centroid-based)

K-Means partitions data into k clusters by minimising within-cluster variance around centroids, iteratively assigning points to the nearest centroid, and updating positions [12]. It is fast, scalable, and widely used, but assumes spherical, equally sized clusters.

The implementation employed dynamic hyperparameter tuning via grid search and fine-tuning. Key search spaces included n_clusters (range 2–20), init methods ('k-means++', 'random'), n_init (10, 20, 50), and max_iter (300, 500), with preprocessing via StandardScaler and optional PCA (retaining 99% variance). Grid search evaluated all combinations using the silhouette score (sil, higher better) and Davies–Bouldin score (db, lower better), selecting the best set via the combined score sil / (db + 1e-12). Fine-tuning refined n_clusters around the best value (±2, minimum 2) using the same formula, updating only if performance improved. Final cluster labels were saved in a new column 'kmeans_cluster'. This process was repeated independently for all 8 datasets.

3.5.4.2. Agglomerative Clustering (Hierarchical)

Agglomerative Clustering builds a hierarchy by merging the closest clusters bottom-up, using linkage criteria to define the distance between clusters [13]. It handles arbitrary shapes, provides dendrograms for visualisation, and does not require a predefined number of clusters.

The implementation used dynamic hyperparameter tuning with sampling for efficiency. Key search spaces included n_clusters (range 2–10), linkage methods ('ward', 'complete', 'average'), and metrics ('euclidean', 'manhattan', 'cosine'), with preprocessing via StandardScaler and PCA (n_components=10). Tuning sampled 1000 rows and evaluated combinations using silhouette score (sil), Calinski-Harabasz index (ch), and Davies-Bouldin index (db), with a combined score (sil + ch_norm + db_norm)/3 where ch_norm = ch / (ch + 1e-6) and db_norm = 1.0 / (1.0 + db). The best set was selected based on the highest combined score. Final cluster labels were saved in a new column 'agg_cluster'. This process was repeated for all 8 datasets.

3.5.4.3. DBSCAN (Density-based)

DBSCAN groups points in high-density regions while marking low-density points as noise, identifying clusters of arbitrary shape without requiring a predefined number of clusters [14]. It is robust to outliers but sensitive to parameter choices.

The implementation used dynamic hyperparameter tuning with sampling for efficiency. Key search spaces included eps (0.3–2.0 in 0.1 increments), min_samples (3, 5, 8, 10, 15), and metrics ('euclidean', 'manhattan', 'cosine'), with preprocessing via StandardScaler and PCA (n_components=10). Tuning evaluated combinations using silhouette score (sil), Calinski-Harabasz index (ch), and Davies-Bouldin index (db), with a combined score (sil + ch_norm + db_norm)/3 where ch_norm = ch / (ch + 1e-6) and db_norm = 1.0 / (1.0 + db). The best parameter set was selected based on the highest combined score. Final cluster labels (noise labelled as -1) were saved in a new column 'dbscan_cluster'. This process was repeated for all 8 datasets.

3.5.4.4. Spectral Clustering (Graph-based)

Spectral Clustering constructs a similarity graph and uses eigenvalues of the graph Laplacian to partition data into non-linear clusters [15]. It excels at capturing complex, non-convex boundaries.

The implementation used dynamic hyperparameter tuning with optional subsampling for efficiency. Key search spaces included n_clusters (range 2 to 6), affinity ('nearest_neighbors'), and assign_labels ('kmeans'), with preprocessing via StandardScaler and PCA (n_components=20). Tuning evaluated combinations using silhouette score (sil), Calinski-Harabasz index (ch), and Davies-Bouldin index (db), with a combined score sil + np.log(ch + 1) - db. The best parameter set was selected based on the highest combined score. Final cluster labels were saved in a new column 'spc_cluster'. This process was repeated for all 8 datasets.

3.5.5. Overall Clustering Pipeline

The pipeline processed each of the 8 datasets uniformly: input DataFrame → preprocessing (scaling + PCA) → hyperparameter tuning (grid search/fine-tuning with combined scoring) → application of the best model → addition of cluster labels column (e.g., 'kmeans_cluster'). This ensured comparability across methods and inputs.

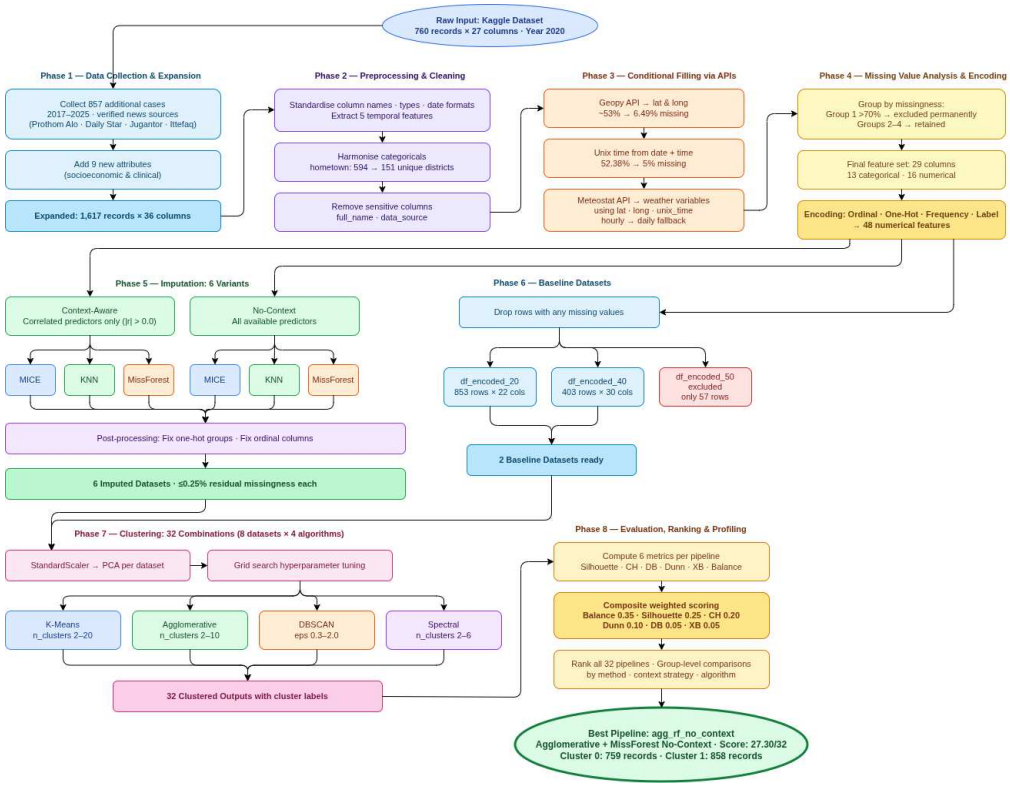

Figure 3.2: Flowchart of the clustering pipeline.

3.5.6. Summary of Clustering Combinations

The table below summarises the 32 combinations (8 inputs × 4 methods), including abbreviations and example metrics (actual values from analysis; e.g., silhouette score).

Input Dataset	Clustering Method	Abbreviation	Imputation Method
df_encoded_20 (853 rows, 22 cols)	K-Means	kmeans_encoded_20	Baseline, no imputation
df_encoded_20	Agglomerative	agg_encoded_20	Baseline, no imputation
df_encoded_20	DBSCAN	dbscan_encoded_20	Baseline, no imputation,
df_encoded_20	SPC (Spectral)	spc_encoded_20	Baseline, no imputation
df_encoded_40 (403 rows, 30 cols)	K-Means	kmeans_encoded_40	Baseline, no imputation
df_encoded_40	Agglomerative	agg_encoded_40	Baseline, no imputation
df_encoded_40	DBSCAN	dbscan_encoded_40	Baseline, no imputation
df_encoded_40	SPC (Spectral)	spc_encoded_40	Baseline, no imputation
df_mice_context	K-Means	kmeans_mice_context	Imputed with MICE
df_mice_context	Agglomerative	agg_mice_context	Imputed with MICE
df_mice_context	DBSCAN	dbscan_mice_context	Imputed with MICE
df_mice_context	SPC (Spectral)	spc_mice_context	Imputed with MICE
df_mice_no_context	K-Means	kmeans_mice_no_context	Imputed with MICE
df_mice_no_context	Agglomerative	agg_mice_no_context	Imputed with MICE
df_mice_no_context	DBSCAN	dbscan_mice_no_context	Imputed with MICE
df_mice_no_context	SPC (Spectral)	spc_mice_no_context	Imputed with MICE
df_knn_context	K-Means	kmeans_knn_context	Imputed with KNN
df_knn_context	Agglomerative	agg_knn_context	Imputed with KNN
df_knn_context	DBSCAN	dbscan_knn_context	Imputed with KNN
df_knn_context	SPC (Spectral)	spc_knn_context	Imputed with KNN
df_knn_no_context	K-Means	kmeans_knn_no_context	Imputed with KNN
df_knn_no_context	Agglomerative	agg_knn_no_context	Imputed with KNN
df_knn_no_context	DBSCAN	dbscan_knn_no_context	Imputed with KNN
df_knn_no_context	SPC (Spectral)	spc_knn_no_context	Imputed with KNN
df_rf_context	K-Means	kmeans_rf_context	Imputed with MissForest
df_rf_context	Agglomerative	agg_rf_context	Imputed with MissForest
df_rf_context	DBSCAN	dbscan_rf_context	Imputed with MissForest
df_rf_context	SPC (Spectral)	spc_rf_context	Imputed with MissForest
df_rf_no_context	K-Means	kmeans_rf_no_context	Imputed with MissForest
df_rf_no_context	Agglomerative	agg_rf_no_context	Imputed with MissForest
df_rf_no_context	DBSCAN	dbscan_rf_no_context	Imputed with MissForest
df_rf_no_context	SPC (Spectral)	spc_rf_no_context	Imputed with MissForest

3.6. Evaluation Metrics and Combined Score Ranking

3.6.1. Overview of Evaluation Approach

To objectively compare the quality of the 32 clustering combinations (8 input datasets × 4 clustering methods), a comprehensive evaluation framework was developed. No single metric fully captures clustering quality, as different indices emphasise various aspects such as cohesion, separation, or compactness. Therefore, five established validation metrics

were selected, supplemented by a newly proposed metric (balance) to address limitations in existing measures, particularly regarding cluster size equity and noise handling.

The goal was to aggregate these metrics into a composite score for ranking, identifying the best pipelines while penalising cases with high geometric quality but poor interpretability (e.g., excessive noise or imbalanced clusters). This multi-metric approach ensured a balanced assessment, prioritising practically useful clusters for suicide pattern analysis. The framework was applied uniformly to all 32 cases, with NaNs handled gracefully in scoring.

3.6.2. Standard Clustering Validation Metrics

Five widely used metrics were employed to evaluate geometric and structural aspects of the clusters. Each was computed for the full dataset (excluding noise labels of -1 in DBSCAN cases, where a mask was applied to focus on clustered points).

Silhouette Score [34]: Measures how similar a point is to its own cluster compared to other clusters, ranging from -1 (poor) to 1 (excellent). It combines intra-cluster cohesion and inter-cluster separation. Formula:

$$s(i) = \frac{b(i) - a(i)}{\max\big(a(i), b(i)\big)}$$

where $a(i)$ is the average distance from i to points in its cluster, and $b(i)$ is the minimum average distance from i to points in another cluster. The overall score is the mean $s(i)$ across all points. Higher values indicate better-defined clusters.

Calinski-Harabasz Index (Variance Ratio Criterion) [35]: Quantifies the ratio of between-cluster dispersion to within-cluster dispersion. Formula:

$$CH = \frac{B/(K-1)}{W/(n-K)}$$

where B is between-cluster sum of squares, W is within-cluster sum of squares, K is number of clusters, and n is number of points. Higher values indicate compact, well-separated clusters.

Davies-Bouldin Index [36]: Measures the average similarity between each cluster and its most similar neighbour, based on scatter and separation. Formula:

$$DB = \frac{1}{K}\sum_{i=1}^{K} \max_{j \neq i} \left(\frac{s_i + s_j}{d_{ij}} \right)$$

where s_i is average intra-cluster distance in cluster i, and d_{ij} is distance between centroids of i and j. Lower values indicate better clustering (less similarity between clusters). In code, it was inverted as $db\_inv = 1/(1+db)$ for higher-better alignment in composites.

Dunn Index [37]: Assesses compactness (max intra-cluster distance) relative to separation (min inter-cluster distance). Formula:

$$D = \frac{\min_{i \neq j} d(c_i, c_j)}{\max_k \delta(c_k)}$$

where $d(c_i, c_j)$ is the minimum distance between points in clusters i and j, and $\delta(c_k)$ is the maximum distance within cluster k. Higher values indicate compact, well-separated clusters.

Xie-Beni Index [38]: Evaluates compactness relative to separation, similar to Davies-Bouldin but with a focus on fuzzy partitioning (adapted for crisp clusters). Formula:

$$XB = \frac{\sum_{i=1}^{n} \sum_{k=1}^{K} (x_i - c_k)^2 / n}{\min_{i \neq j} \|c_i - c_j\|^2}$$

where c_k are centroids. Lower values indicate better clustering. In code, it was inverted as $xb\_inv = 1/(1+xb)$ for higher-better composites.

These metrics were chosen for their complementary strengths in assessing geometric validity, though they do not directly penalise size imbalance or excessive noise.

3.6.3. Proposed Balance Metric

Standard metrics often overlook cluster size equity, leading to high scores for configurations with imbalanced distributions (e.g., one large cluster and many singletons) or excessive noise (e.g., DBSCAN flagging 99% as outliers). To address this, a new balance metric was proposed to promote interpretable, usable clusters by rewarding even point distribution and penalising over-segmentation or high noise.

The metric is calculated as follows:

Let L be the array of cluster labels (excluding noise/-1 for DBSCAN), K the number of unique clusters ($\geq$2, else score=0), c_k the size of cluster k, $n = \sum c_k$, and $p_k = c_k/n$ (proportions).

First, the base balance (evenness):

$$\text{base} = 1 - \sqrt{\frac{1}{K} \sum_{k=1}^{K} \left(p_k - \frac{1}{K} \right)^2}$$

This is 1 for perfectly equal sizes and decreases with imbalance (standard deviation of proportions).

Finally with K penalty:

$$\text{balance} = \text{base} \times \frac{1}{\sqrt{K}}$$

The penalty $1/\sqrt{K}$ discourages excessive clusters (e.g., many tiny groups). The score ranges ~0–1, with higher values for balanced, moderately segmented clusters.

This metric is crucial for suicide data analysis, where imbalanced clusters hinder pattern profiling (e.g., rare subgroups like specific professions ignored if tiny). It adds practical interpretability to geometric metrics, making it a novel contribution for applications requiring actionable insights.

3.6.4. Composite Scoring and Ranking Framework

To integrate the six metrics into a single ranking, a weighted rank-based aggregation was designed. This approach handles differing scales, directions (higher/lower better), and cases where metrics are undefined (NaN), while prioritizing interpretability through a high weight on the balance metric.

For each of the 32 cases, metrics were computed. Per metric, cases were ranked descending (higher-better for silhouette, Calinski-Harabasz, Dunn, balance, and inverted Davies-Bouldin/Xie-Beni; NaNs ranked last). Points were assigned: n – rank position (n=32, top=32, NaN=0).

The composite score for a case was:

$$\text{score} = \sum_{m=1}^{6} \omega_m \times (n - r_m)$$

where ω_m is the weight of metric m, and r_m is its rank (0 for NaN).

Weights were: balance (0.35), silhouette (0.25), Calinski-Harabasz (0.20), Dunn (0.10), Davies-Bouldin (0.05), Xie-Beni (0.05). Cases were sorted by descending score for the final ranking.

Weights were selected empirically by testing several combinations and manually inspecting the resulting top-ranked clusters for interpretability, noise levels, size balance, and logical distinguishability of patterns. The final weights (shown in Table 3.1) were chosen as the combination that consistently produced the most meaningful, well-distributed, and low-noise clusters suitable for suicide data pattern discovery.

Metric	Weight	Rationale
balance	0.35	Highest priority for size equity and noise control
silhouette	0.25	Core measure of cohesion/separation
calinski_harabasz	0.20	Strong variance-based separation indicator
dunn	0.10	Distance-based compactness/separation
davies_bouldin	0.05	Cluster similarity (inverted)
xie_beni	0.05	Compactness relative to separation (inverted)

Table 3.1: Weights for composite score

And this composite score, since we have all 32 cases, so 32 is the highest score one case can achieve while the lowest can be down to 1. So the cases closest to 32 is performing better and closest to 1 is performing worst.

3.6.5. Group-Level Comparisons

To compare across imputation strategies, predefined groups were formed: no_context_dfs (all no-context imputed), context_dfs (all context-aware imputed), no_imputed_dfs (baseline encoded), mice_dfs (MICE variants), rf_dfs (MissForest variants), knn_dfs (KNN variants), and algorithm-specific groups (agg_dfs, dbscan_dfs, kmeans_dfs, spc_dfs). Group averages were computed per metric using summation and division, with NaNs skipped, for visualisation and cross-analysis. Metrics were normalised via min-max scaling (per metric across all cases) for comparative graphing, ensuring fair visual representation.

3.6.6. Additional Analysis for Best Cases

Post-ranking, top cases were selected for decoding and profiling. Label-encoded features were decoded using saved JSON files (from preprocessing) to map integers back to original categories, enabling cluster interpretation, visualisation, and real-world pattern discovery.